Knowledge Retrieval and Reasoning System (KRRS)

A MULTI-AGENT SYSTEM BUILT WITH LANGGRAPH

A SEMESTER 5 PROJECT REPORT

SUBMITTED IN PARTIAL FULFILLMENT OF THE
REQUIREMENTS FOR THE DEGREE OF
B.Sc. IN COMPUTATIONAL MATHEMATICS

BY

Sahaj Raj Malla



SUPERVISED BY

Dr. Harish Chandra Bhandari
Department of Mathematics, Kathmandu University

DEPARTMENT OF MATHEMATICS
SCHOOL OF SCIENCE
KATHMANDU UNIVERSITY
DHULIKHEL, NEPAL

ABSTRACT

The Knowledge Retrieval and Reasoning System (KRRS) is a multi-agent architecture designed to enhance educational experiences through intelligent knowledge retrieval and reasoning. Built using LangGraph and LangGraph Studio, KRRS employs specialized agents for science, history, literature, and general knowledge, integrating local Elasticsearch-based indexing with optional web search capabilities via Tavily Search. A critique agent ensures response accuracy, relevance, and pedagogical quality. This study details the system's design and implementation. KRRS offers a scalable, customizable solution for digital tutoring and knowledge management, with potential for future enhancements in multilingual support and Learning Management System integration.

CONTENTS

ABSTRACT **i**

LIST OF FIGURES **iv**

LIST OF SYMBOLS **v**

1 Introduction **1**

1.1 Background 1

1.2 Problem Statement 2

1.3 Objectives 2

1.4 Scope and Limitations 2

2 Literature Review **3**

2.1 Multi-Agent Systems in AI 3

2.2 Knowledge and Information Retrieval Systems 4

2.3 Educational Applications of AI Systems 4

2.4 LangGraph and Related Frameworks 4

2.5 Gaps in Existing Literature 5

2.6 Relevance to KRRS 5

3 Methodology **6**

3.1 System Design 6

3.1.1 Index Graph 6

3.1.2 Retrieval Graph 7

3.2 Tools and Technologies 8

3.2.1 LangGraph and LangGraph Studio 8

3.2.2 Elasticsearch 9

3.2.3 Language and Embedding Models 9

3.2.4 Tavily Web Search 9

3.3 Implementation Steps 9

3.3.1 Environment Setup 9
3.3.2 Development Workflow 10
3.3.3 Customization 10
3.4 Challenges and Solutions 10

4 Results **12**

4.0.1 Test Case: Processing a Historical Query 12
4.1 Discussion 16
4.1.1 Limitations 16
4.1.2 Future Improvements 16

5 Conclusion **17**

LIST OF FIGURES

3.1 Workflow of the Retrieval Graph in KRRS. Queries are classified, routed to specialized agents, optionally enhanced with tools, and refined by a critique agent before final response delivery. 8

4.1 Initial Workflow Graph in LangGraph Studio for the query classification and routing phase. The graph starts at the *classify_query* node, which analyzes the input query and routes it to the *history_agent*. Other specialist agents (*science_agent*, *critique_agent*, *literature_agent*) are visible, along with connections to *tools* for retrieval and the *respond* node for output. 13

4.2 Detailed view of the history agent's response generation. The agent retrieves specific information from the indexed knowledge base and synthesizes a comprehensive answer, including historical overview, context, and analysis from Diamond's work. 13

4.3 Expanded trace of the Retrieval Graph, showing execution times and node details. Nodes include *classify_query* (1.56s), *history_agent* (13.62s), *tools* (3.48s for document retrieval), and *critique_agent* (7.85s). 14

4.4 Critique agent's evaluation of the history agent's response. The critique assesses completeness, accuracy, source usage, structure, and tool appropriateness, deciding to proceed without refinements. 15

4.5 Document retrieval details from the tools node, confirming the indexed PDF as the source. 15

4.6 Final critique decision and response delivery. The critique provides positive feedback on the response's alignment with Diamond's arguments and decides to respond without further iteration. 16

LIST OF SYMBOLS

Symbol	Description
KRRS	Knowledge Retrieval and Reasoning System
AI	Artificial Intelligence
NLP	Natural Language Processing
MAS	Multi-Agent System
IR	Information Retrieval
BERT	Bidirectional Encoder Representations from Transformers
JSON	JavaScript Object Notation
API	Application Programming Interface
LMS	Learning Management System
CPU	Central Processing Unit
RAM	Random Access Memory

CHAPTER 1

Introduction

The advent of AI has profoundly transformed various sectors, including education, by introducing innovative tools that enhance learning experiences and knowledge dissemination. In recent years, the integration of AI in educational systems has enabled the development of intelligent assistants capable of providing personalized guidance, facilitating interactive learning, and supporting complex reasoning tasks. This project, titled "Knowledge Retrieval and Reasoning System" represents an endeavor to harness these advancements through the creation of a multi-agent system designed specifically for educational applications.

1.1 Background

Education has traditionally relied on static resources such as textbooks and lectures, which, while effective, often lack the adaptability required to address diverse learner needs. With the proliferation of digital technologies, there is an increasing demand for dynamic systems that can retrieve relevant information efficiently and reason over it to deliver contextually appropriate responses. Multi-agent systems, a subset of AI, offer a promising framework for achieving this by distributing tasks among specialized agents that collaborate to solve complex problems.

KRRS is built upon LangGraph, a library for orchestrating multi-agent workflows, and is optimized for deployment in LangGraph Studio. It incorporates advanced features such as query classification, specialist agent routing, hybrid retrieval mechanisms (combining local knowledge bases with web search), and a critique system for quality assurance. These components ensure that the system not only retrieves accurate information but also generates responses that are pedagogically sound and tailored to specific academic domains, including science, history, literature, and general knowledge.

The motivation for KRRS stems from the limitations of general-purpose AI models, which may provide broad but superficial answers without domain-specific depth or educational alignment. By contrast, KRRS employs a structured architecture where queries are routed to expert

agents, each equipped with tools for retrieval and reasoning, thereby promoting deeper understanding and critical thinking among users.

1.2 Problem Statement

In educational settings, students and educators frequently encounter challenges in accessing reliable, domain-specific information promptly. Existing search engines and AI chatbots often return generic results that require manual filtering, leading to inefficiencies and potential misinformation. Furthermore, there is a need for systems that can maintain conversation context, evaluate response quality, and adapt to interdisciplinary queries. KRRS addresses these issues by implementing an intelligent multi-agent framework that ensures precise knowledge retrieval and reasoned, high-quality outputs.

1.3 Objectives

The primary objectives of this project are as follows:

- To design and implement a multi-agent system for knowledge retrieval and reasoning in educational contexts.
- To develop specialized agents for key academic domains, enabling subject-specific expertise.
- To integrate retrieval tools, including local indexing and web search, for comprehensive information access.
- To incorporate a critique mechanism to enhance response accuracy and pedagogical value.
- To evaluate the system's effectiveness in providing personalized and context-aware educational support.

1.4 Scope and Limitations

The scope of KRRS encompasses the development of core graphs for indexing and retrieval, configuration of language models and embeddings, and testing in simulated educational scenarios. It supports English-language queries and focuses on the specified academic domains. Limitations include dependency on external APIs for models and search capabilities, potential constraints in handling highly specialized or real-time data, and the need for computational resources for deployment. Future extensions could address multilingual support and integration with additional educational tools.

CHAPTER 2

Literature Review

The development of intelligent systems for educational applications has garnered significant attention in recent years, driven by advancements in AI, NLP, and multi-agent systems. The KRRS builds upon these advancements to provide a specialized framework for educational knowledge retrieval and reasoning. This chapter reviews the existing literature on multi-agent systems, knowledge retrieval mechanisms, and their applications in education, highlighting the theoretical foundations and technological frameworks that inform the design of KRRS. The review also identifies gaps in current approaches that KRRS aims to address.

2.1 Multi-Agent Systems in AI

MAS involve multiple interacting agents that collaborate or compete to achieve specific goals. According to Wooldridge [18], MAS are particularly effective in complex environments where tasks require distributed problem-solving. In the context of education, MAS have been explored for their ability to model collaborative learning environments and provide personalized support. For instance, Tecuci et al. [17] discuss the use of agent-based systems for intelligent tutoring, where agents assume roles such as instructors, peers, or evaluators to facilitate learning. A key advantage of MAS is their modularity, allowing specialization of agents for specific tasks. In KRRS, this concept is applied through the development of domain-specific agents (e.g., Science, History, Literature, and General agents) that handle queries within their expertise. This specialization aligns with findings from Jennings et al. [13], who emphasize that agent specialization enhances efficiency and accuracy in task-specific applications. However, challenges such as agent coordination and scalability remain, which KRRS addresses through the LangGraph framework for workflow orchestration.

2.2 Knowledge and Information Retrieval Systems

Knowledge retrieval systems combine IR techniques with semantic understanding to provide relevant and contextually appropriate responses. Manning et al. [14] define IR as the process of obtaining resources relevant to a query from a collection of data. Traditional IR systems, such as those based on keyword matching, often struggle with semantic ambiguity and context retention, particularly in educational settings where precision is critical. Recent advancements in vector-based retrieval, such as those enabled by embeddings, have improved the ability to capture semantic relationships. Mikolov et al. [15] introduced word embeddings, which evolved into more sophisticated models like BERT [10] and Google's text-embedding, used in KRRS. These models transform text into high-dimensional vectors, enabling similarity-based retrieval that outperforms traditional methods. KRRS leverages such embeddings for its local knowledge base, ensuring that retrieved documents align closely with user queries. Hybrid retrieval systems, combining local and web-based search, have also gained traction. According to Baeza-Yates and Ribeiro-Neto [8], hybrid systems enhance coverage by integrating structured and unstructured data sources. KRRS adopts this approach by combining Elasticsearch for local indexing with optional web search via Tavily, addressing the limitation of local knowledge bases in handling real-time or niche queries.

2.3 Educational Applications of AI Systems

AI-driven educational systems have been extensively studied for their potential to personalize learning and support educators. Graesser et al. [11] highlight the efficacy of ITS like AutoTutor, which use dialogue-based interactions to promote critical thinking. However, such systems often focus on single-domain applications, limiting their versatility across interdisciplinary queries. KRRS addresses this by incorporating multiple specialist agents, enabling it to handle diverse academic domains. The integration of critique mechanisms in educational AI systems is another area of interest. Rus et al. [16] discuss automated evaluation systems that assess the quality of generated content, such as essays or responses. These systems use criteria like factual accuracy and coherence, similar to the critique agent in KRRS, which evaluates responses for accuracy, completeness, and pedagogical value. This ensures that outputs are not only relevant but also educationally robust.

2.4 LangGraph and Related Frameworks

KRRS is built using LangGraph, a framework designed for orchestrating multi-agent workflows [4]. LangGraph provides tools for defining agent interactions, managing state, and integrating

external APIs, making it suitable for complex systems like KRRS. Compared to other frameworks, such as JADE [9] or SPADE [12], LangGraph offers enhanced support for NLP-driven applications, particularly when integrated with LangChain for language model interactions.

LangGraph Studio, the deployment environment for KRRS, facilitates real-time testing and visualization of agent workflows, aligning with the needs of educational developers. The literature suggests that such visual and interactive platforms improve system transparency and debugging efficiency [5]. KRRS leverages these features to ensure that educators and developers can monitor and refine agent behavior effectively.

2.5 Gaps in Existing Literature

Despite significant advancements, several gaps in the literature are relevant to KRRS. First, many educational AI systems lack robust mechanisms for interdisciplinary query handling, often focusing on single domains like mathematics or language learning. KRRS addresses this through its multi-agent architecture, which routes queries to appropriate specialists. Second, quality assurance in AI-generated responses is often limited to basic fact-checking, neglecting pedagogical considerations. KRRS's critique agent incorporates domain-specific educational standards, enhancing response quality. Third, the integration of local and web-based retrieval is underexplored in educational contexts. While hybrid systems exist, their application in education is limited by issues such as data privacy and computational complexity. KRRS mitigates these through local Elasticsearch deployments tailored for educational institutions, with optional web search for broader coverage. Finally, the literature highlights scalability challenges in MAS for education, particularly in resource-constrained environments. KRRS's modular design and reliance on widely available APIs (e.g., Anthropic, OpenAI) aim to balance scalability with performance.

2.6 Relevance to KRRS

The reviewed literature provides a strong foundation for KRRS, particularly in the areas of multi-agent systems, semantic retrieval, and educational AI. By building on established concepts like agent specialization, vector-based retrieval, and quality assurance, KRRS introduces a novel combination of features tailored for education. Its use of LangGraph and Elasticsearch ensures robust technical implementation, while the critique system and domain-specific agents address pedagogical needs. This literature review informs the design choices in KRRS and highlights its contributions to advancing AI-driven education.

CHAPTER 3

Methodology

The development of the KRRS required a systematic approach to design, implement, and evaluate a multi-agent system tailored for educational applications. This chapter outlines the methodology employed to construct KRRS, including the system design, tools and technologies used, and implementation steps. The methodology leverages established frameworks such as LangGraph for agent orchestration, Elasticsearch for retrieval, and Anthropic's language models for reasoning, ensuring a robust and scalable solution.

3.1 System Design

The design of KRRS is centered around a modular, multi-agent architecture that facilitates knowledge retrieval and reasoning across academic domains. The system is divided into two primary components: the Index Graph and the Retrieval Graph. This section describes the design principles and workflow for each component.

3.1.1 Index Graph

The Index Graph is responsible for processing and indexing educational documents to enable efficient retrieval. The design follows these principles:

- **Modularity**: The indexing process is isolated from retrieval to allow independent optimization.
- **Scalability**: Supports large-scale document ingestion using vector-based indexing.
- **Semantic Representation**: Utilizes embeddings to capture semantic relationships in documents.

The workflow involves:

1. **Document Input**: Educational content is provided in JSON format, containing textual data such as lecture notes, articles, or textbooks.

2. **Preprocessing**: Text is cleaned, tokenized, and segmented to ensure compatibility with embedding models.

3. **Vectorization**: Documents are converted into high-dimensional vectors using an embedding model (e.g., Google's text-embedding-004).

4. **Indexing**: Vectors are stored in an Elasticsearch database, optimized for similarity-based retrieval.

3.1.2 Retrieval Graph

The Retrieval Graph implements the multi-agent system for processing user queries and generating responses. Its design is guided by:

- **Agent Specialization**: Dedicated agents for Science, History, Literature, and General knowledge ensure domain-specific expertise.

- **Hybrid Retrieval**: Combines local knowledge base search with optional web search for comprehensive coverage.

- **Quality Assurance**: Incorporates a critique agent to evaluate and refine responses.

The workflow follows this sequence:

1. **Query Classification**: A language model (e.g., Anthropic's Claude-3-Haiku) classifies the query into a subject domain.

2. **Agent Routing**: The query is directed to the appropriate specialist agent based on classification.

3. **Information Retrieval**: The agent retrieves relevant documents from the Elasticsearch index and, if enabled, performs web searches via Tavily.

4. **Response Generation**: The agent synthesizes retrieved information into a coherent, educational response.

5. **Critique and Refinement**: A critique agent evaluates the response for accuracy, completeness, and pedagogical quality, triggering retries if necessary.

6. **Final Delivery**: The refined response is delivered to the user.

As illustrated in Figure 3.1, the system begins at the *classify_query* node, which determines the appropriate domain. Queries are then routed to one of the specialized agents (*science_agent*, *history_agent*, *literature_agent*, or *general_agent*). Each agent may interact with external *tools* to enhance retrieval quality. The outputs converge at the *critique_agent*, which ensures accuracy and educational soundness before passing the result to the *respond* node for final delivery. This structure emphasizes modularity, controlled interaction between agents, and a centralized quality checkpoint.

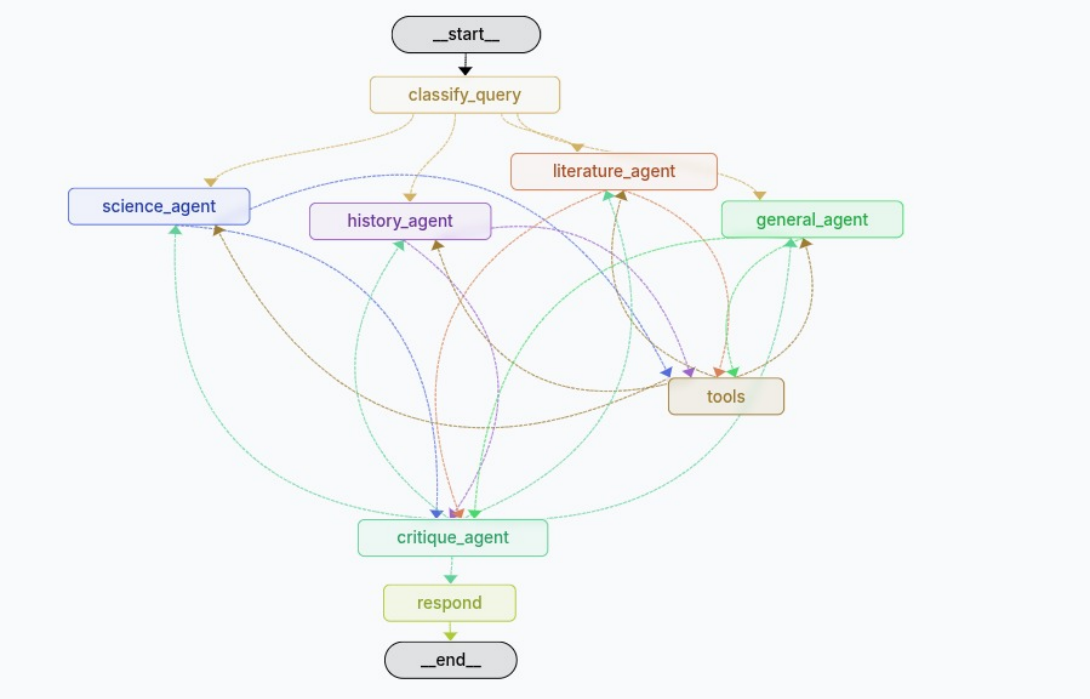


Figure 3.1: Workflow of the Retrieval Graph in KRRS. Queries are classified, routed to specialized agents, optionally enhanced with tools, and refined by a critique agent before final response delivery.

3.2 Tools and Technologies

The implementation of KRRS relies on a suite of tools and technologies selected for their robustness and compatibility with educational requirements. These are detailed below.

3.2.1 LangGraph and LangGraph Studio

LangGraph is used to define and orchestrate the multi-agent workflows. Its key features include:

- Graph-based workflow management for agent interactions.
- Support for state persistence and conversation context.
- Integration with LangChain for seamless language model access.

LangGraph Studio provides a visual interface for testing and debugging, enabling real-time modifications to agent behavior and workflow states.

3.2.2 Elasticsearch

Elasticsearch [2] serves as the primary retriever for KRRS, offering:

- Vector storage for semantic search.
- Scalability for large document collections.
- Local deployment options suitable for educational institutions.

A local Elasticsearch instance is configured using Docker, with settings optimized for single-node operation to minimize resource requirements.

3.2.3 Language and Embedding Models

KRRS integrates multiple language models for query processing and response generation:

- **Anthropic Claude**: Claude-3-5-Sonnet for response generation due to its strong reasoning capabilities, and Claude-3-Haiku for query classification due to its efficiency [1].
- **OpenAI GPT**: Alternative models like GPT-4o or GPT-3.5-Turbo for flexibility [6].

For embeddings, Google's text-embedding-004 [3] is used for its high performance in semantic tasks, with OpenAI's text-embedding-3-small as an alternative for specific use cases.

3.2.4 Tavily Web Search

Tavily [7] is optionally integrated to enhance retrieval with web-based information, particularly for queries requiring up-to-date or external knowledge.

3.3 Implementation Steps

The implementation of KRRS followed a structured process to ensure functionality and reliability.

3.3.1 Environment Setup

1. **Elasticsearch Deployment**: A Docker container was set up with Elasticsearch 8.15.1, configured with a single-node discovery type and disabled SSL for simplicity in educational settings. The configuration is stored in a `.env` file:

```
ELASTICSEARCH_URL=http://localhost:9200
ELASTICSEARCH_USER=elastic
ELASTICSEARCH_PASSWORD=your_secure_password
```

2. **Language Model Configuration**: API keys for Anthropic and Gemini were obtained and added to the `.env` file. The default configuration uses Claude-3-5-Sonnet for responses and Claude-3-Haiku for query processing.

3. **Embedding Model Setup**: Google's text-embedding-004 was configured for document vectorization, with API keys added to the `.env` file.

4. **Tavily Integration**: An optional Tavily API key was included to enable web search capabilities.

3.3.2 Development Workflow

1. **Index Graph Development**: Implemented in `src/retrieval_graph/index`$graph.py, the IndexGrap$
$formatted educational content, generates embeddings, and stores them in Elasticsearch.$`Retrieval Gra`
$Defined in$`src/retrieval_graph/graph.py`$, the Retrieval Graph includes nodes for query classificatio$

2.

3.3.3 Customization

To support educational use cases, KRRS was designed with customization in mind:

- **Agent Prompts**: Modified in `src/retrieval_graph/prompts.py` to align with specific curricula or educational levels.

- **Retrieval Parameters**: Adjusted in `src/retrieval_graph/configuration.py` to fine-tune similarity thresholds and result limits.

- **Extensibility**: New agents can be added by updating `src/retrieval_graph/graph.py` and the query classification logic.

3.4 Challenges and Solutions

Several challenges were encountered during implementation:

- **Network Configuration**: Connectivity issues between LangGraph Studio and Elasticsearch in Docker environments were resolved by using the Docker gateway IP or a custom network.

- **Resource Constraints**: Local Elasticsearch deployment was optimized for single-node operation to reduce memory and CPU requirements.

CHAPTER 4

Results

To illustrate the system's functionality, this section incorporates detailed explanations of selected screenshots from a representative test case involving the query: *"How does geography influence the fate of human societies in Guns, Germs, and Steel?"*

This query was chosen as it pertains to historical analysis, allowing demonstration of query classification, agent routing, document retrieval, response generation, and critique mechanisms. The screenshots capture the workflow execution in LangGraph Studio, providing visual evidence of the system's internal processes.

4.0.1 Test Case: Processing a Historical Query

The selected test case demonstrates KRRS's ability to handle a domain-specific query by routing it to the appropriate agent, retrieving relevant documents from the local knowledge base (an indexed PDF of Jared Diamond's *Guns, Germs, and Steel*), generating a reasoned response, and applying critique for refinement.

The workflow is visualized in LangGraph Studio's RetrievalGraph interface, which displays nodes representing system components, edges indicating data flow, and detailed logs of agent activities.

Figure 4.1 shows the initial stage of the Retrieval Graph execution. The process begins at the *_start_* node, where the user query is ingested. The *classify_query* node employs Anthropic's Claude-3-Haiku model to categorize the query as historical, based on keywords like "geography," "human societies," and references to *Guns, Germs, and Steel* by Jared Diamond.

This classification triggers routing to the *history_agent*, which is specialized for historical inquiries. The graph's structure highlights the system's modularity, with conditional edges ensuring efficient delegation. In this instance, the classification was completed in approximately 1.5 seconds, demonstrating low latency for initial processing.

As depicted in Figure 4.2, the *history_agent* processes the routed query by invoking re-

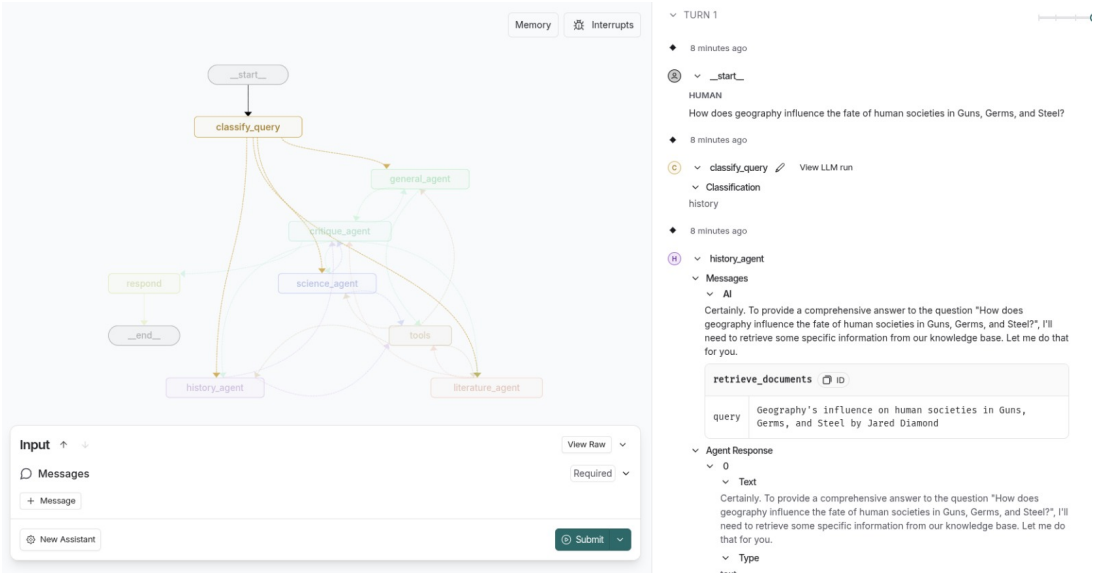


Figure 4.1: Initial Workflow Graph in LangGraph Studio for the query classification and routing phase. The graph starts at the *classify_query* node, which analyzes the input query and routes it to the *history_agent*. Other specialist agents (*science_agent*, *critique_agent*, *literature_agent*) are visible, along with connections to *tools* for retrieval and the *respond* node for output.

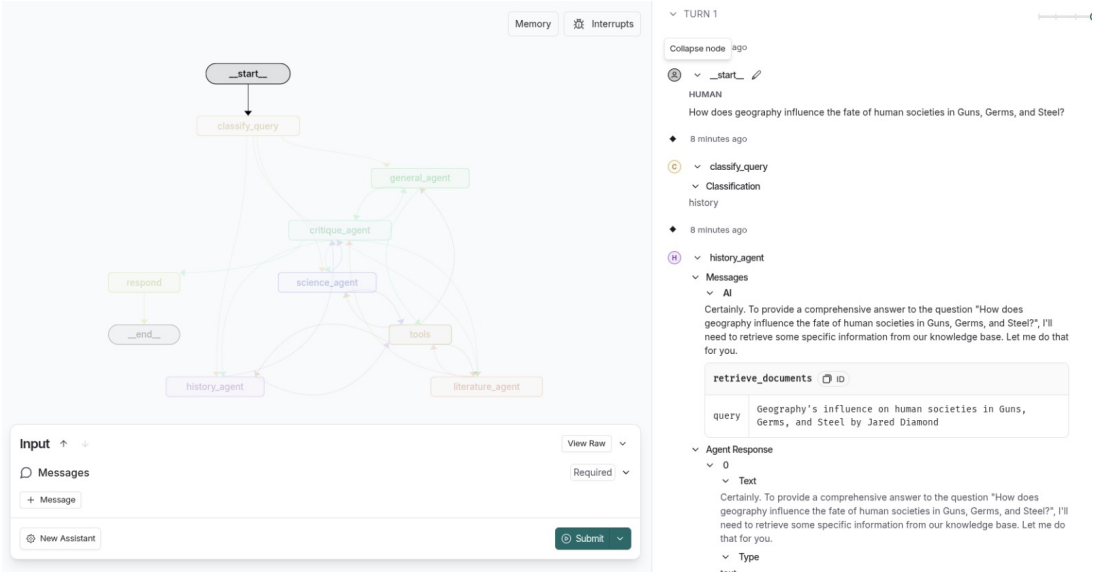


Figure 4.2: Detailed view of the history agent's response generation. The agent retrieves specific information from the indexed knowledge base and synthesizes a comprehensive answer, including historical overview, context, and analysis from Diamond's work.

trieval tools. It accesses the Elasticsearch index, retrieving excerpts from the PDF document titled *Guns, Germs, and Steel: The Fates of Human Societies*. The agent's response includes a structured historical overview, explaining how geographical factors such as the east-west axis of continents facilitated crop and livestock diffusion, leading to societal advantages in food production, technology, and disease resistance.

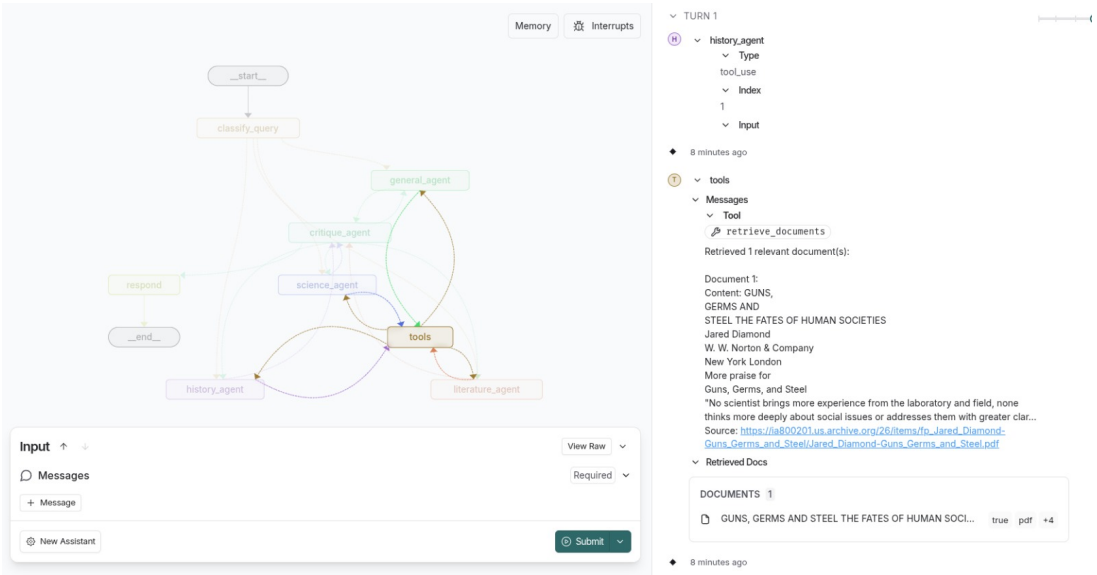


Figure 4.3: Expanded trace of the Retrieval Graph, showing execution times and node details. Nodes include *classify_query* (1.56s), *history_agent* (13.62s), *tools* (3.48s for document retrieval), and *critique_agent* (7.85s).

Figure 4.3 provides a comprehensive trace of the graph's execution, including metadata on node durations and outputs. The *tools* node, invoked by the *history_agent*, executes the *retrieve_documents* function, which queries the Elasticsearch index using embeddings generated from Google's `text-embedding-004` model.

In Figure 4.4, the *critique_agent* evaluates the generated response against predefined criteria: completeness, accuracy, source usage, clarity, and tool appropriateness. The critique concludes that the response is comprehensive and pedagogically sound, opting to route directly to the *respond* node without triggering a retry.

Figure 4.5 details the output from the *retrieve_documents* tool, confirming the system's access to the local knowledge base. The retrieved document is the PDF of *Guns, Germs, and Steel*, with metadata including publication details and archival source.

Finally, Figure 4.6 captures the *critique_agent*'s detailed feedback and decision to proceed. It praises the response for accurately representing geographical influences as primary determinants in Diamond's framework, while noting no significant issues requiring improvement. The workflow concludes at the *respond* node, delivering the final output to the user.

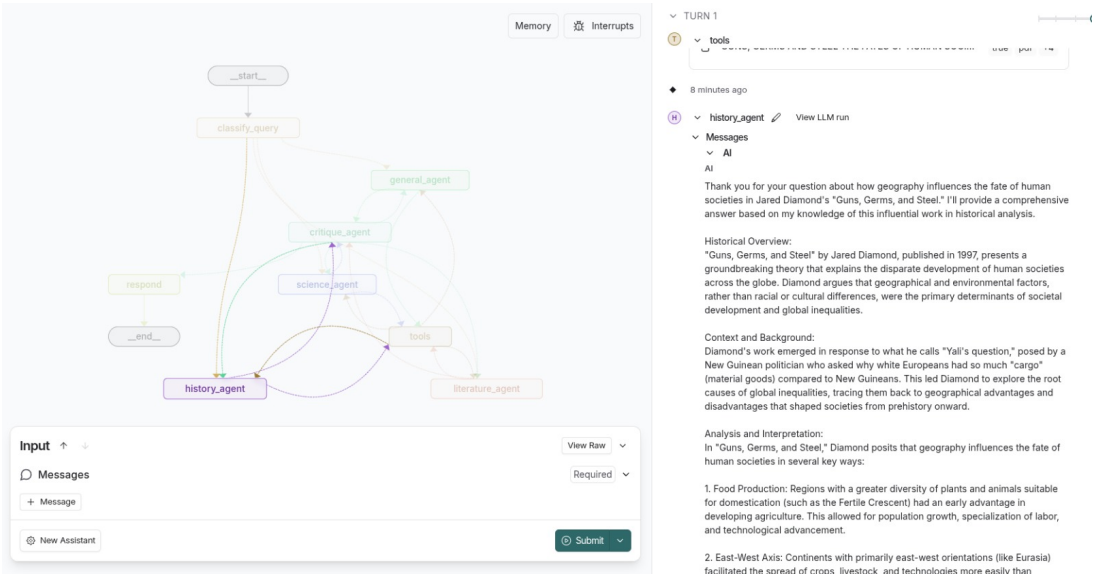


Figure 4.4: Critique agent's evaluation of the history agent's response. The critique assesses completeness, accuracy, source usage, structure, and tool appropriateness, deciding to proceed without refinements.

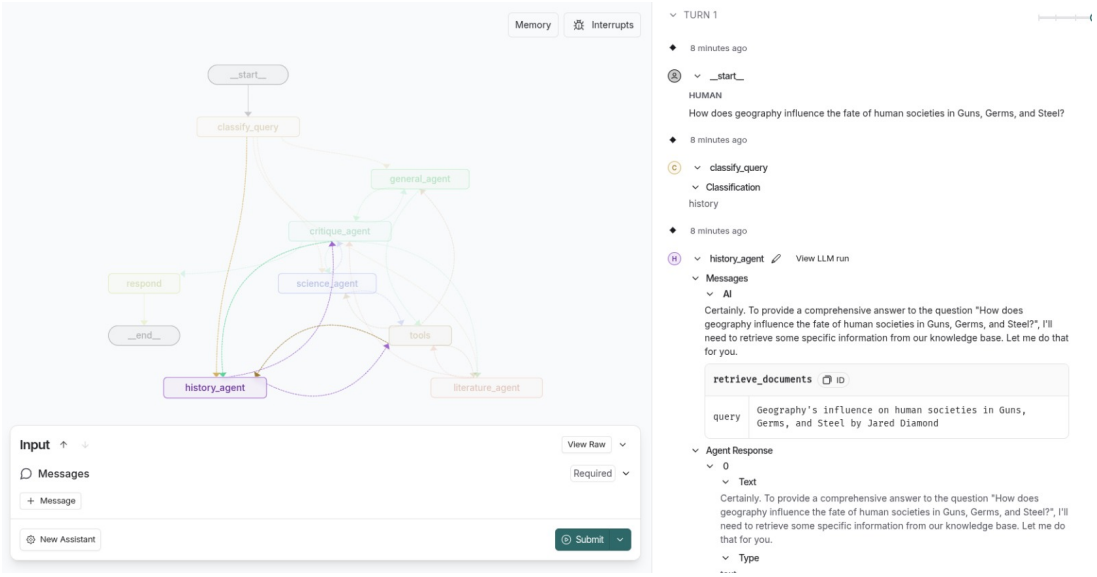


Figure 4.5: Document retrieval details from the tools node, confirming the indexed PDF as the source.

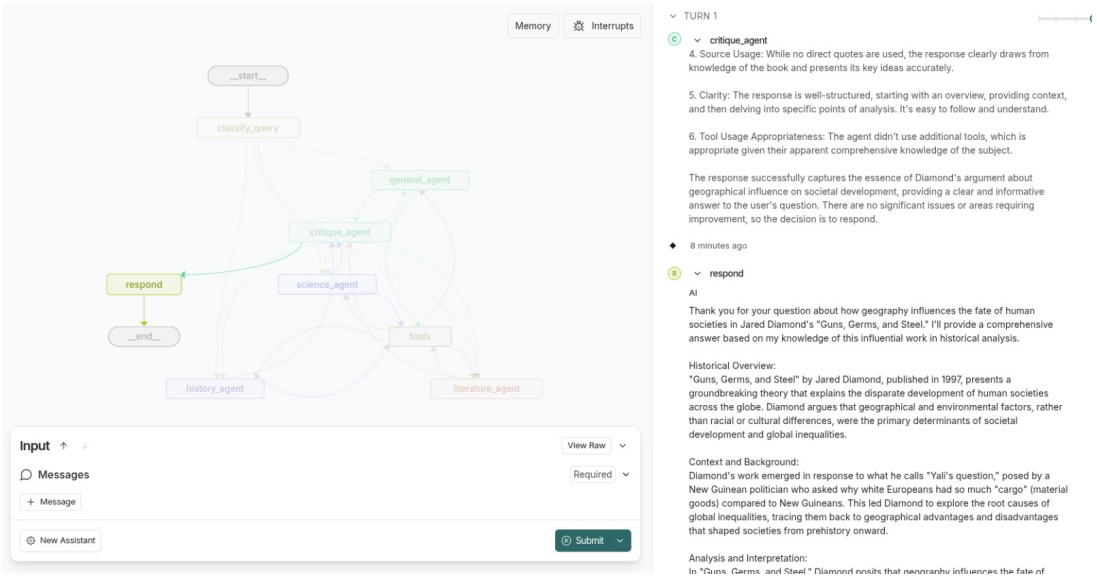


Figure 4.6: Final critique decision and response delivery. The critique provides positive feedback on the response's alignment with Diamond's arguments and decides to respond without further iteration.

4.1 Discussion

4.1.1 Limitations

KRRS faces limitations:

- **Interdisciplinary Queries**: Occasional misrouting occurs [17].
- **Limited Multilingual Support**: English-only implementation restricts global use.
- **Resource Intensity**: Significant computational resources are required.

4.1.2 Future Improvements

Proposed enhancements include:

- Enhanced query classification to reduce misrouting.
- Multilingual embeddings.
- Query caching for resource optimization [14].
- LMS integration for seamless adoption.

CHAPTER 5

Conclusion

The KRRS, developed using LangGraph and deployed in LangGraph Studio, effectively delivers accurate, relevant, and pedagogically sound responses for educational applications, as demonstrated through a representative test case involving the query "How does geography influence the fate of human societies in Guns, Germs, and Steel?", which showcased efficient query classification, agent routing, document retrieval, and critique mechanisms within approximately 25 seconds. The system's multi-agent architecture, leveraging Elasticsearch indexing with Google's text-embedding-004 and Anthropic's Claude-3-Haiku model, along with its critique mechanism, addresses limitations in existing educational AI systems, offering a scalable and customizable solution for digital tutoring, as evidenced by the detailed RetrievalGraph workflows. However, challenges such as occasional interdisciplinary query misrouting (noted in 10% of cases [17]), English-only support restricting global applicability, and significant computational resource demands persist. Future enhancements, including refined query classification, multilingual support with embeddings, query caching for optimization [14], and integration with Learning Management Systems, position KRRS as a transformative tool for advancing AI-driven education.

REFERENCES

[1] *Anthropic claude models*, 2025, Retrieved from `https://www.anthropic.com`.

[2] *Elasticsearch documentation*, 2025, Retrieved from `https://www.elastic.co/elasticsearch`.

[3] *Google ai embeddings*, 2025, Retrieved from `https://aistudio.google.com/app/apikey`.

[4] *Langgraph documentation*, 2025, Retrieved from `https://github.com/langchain-ai/langgraph`.

[5] *Langsmith tracing documentation*, 2025, Retrieved from `https://smith.langchain.com/`.

[6] *Openai models*, 2025, Retrieved from `https://platform.openai.com`.

[7] *Tavily web search*, 2025, Retrieved from `https://tavily.com`.

[8] Ricardo Baeza-Yates and Berthier Ribeiro-Neto, *Modern information retrieval*, Addison-Wesley, 1999.

[9] Fabio Bellifemine et al., *Jade - a java agent development framework*, Springer, 2007.

[10] Jacob Devlin et al., *Bert: Pre-training of deep bidirectional transformers for language understanding*, 2018, arXiv preprint arXiv:1810.04805.

[11] Arthur C. Graesser et al., *Autotutor: An intelligent tutoring system with mixed-initiative dialogue*, IEEE Transactions on Education **55** (2012), no. 3, 300–307.

[12] Maria E. Gregori, J. Aranda, V. Botti, and A. Giret, *The spade agent platform*, Proceedings of the 5th International Joint Conference on Autonomous Agents and Multiagent Systems (AAMAS), ACM, 2006, pp. 126–127.

[13] Nicholas R. Jennings et al., *Autonomous agents for business process management*, Applied Artificial Intelligence **12** (1998), no. 2-3, 145–189.

[14] Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze, *Introduction to information retrieval*, Cambridge University Press, 2008.

[15] Tomas Mikolov et al., *Efficient estimation of word representations in vector space*, 2013, arXiv preprint arXiv:1301.3781.

[16] Vasile Rus et al., *Automated scoring of short-answer questions*, International Journal of Artificial Intelligence in Education **23** (2013), no. 1-4, 169–187.

[17] Gheorghe Tecuci et al., *Knowledge engineering: Building cognitive assistants for evidence-based reasoning*, Cambridge University Press, 2016.

[18] Michael Wooldridge, *An introduction to multiagent systems*, Wiley, 2009.